# Formal Symbols in $\mathcal{UD}$

Brian Beckman
Feb 2026

In many tensorial calculations in $\mathcal{UD}$, we see formal symbols like `𝚣` or extended formal symbols like `𝛌123`. Formal symbols, extended or not, are like ordinary symbols, except they're protected: you can't assign values to them without purposefully unprotecting them. That feature saves accidental assignment to indices on tensors, which must remain fully symbolic for $\mathcal{UD}$ to do its work.

Mathematica directly supports "Pure System" formal symbols like `𝛂`, consisting of exactly one formal character. All such formal characters are listed below. $\mathcal{UD}$ needs extended formal symbols, which have some non-formal characters after an initial formal character, for `Unique` protected symbols, guaranteed not to collide with other symbols of any kind.

Wolfram does not offer `FormalSymbolQ`, so we implement it here. Extended formal symbols are a $\mathcal{UD}$ invention: Wolfram knows nothing about them, so we implement a constructor, `CreateExtendedFormal`, and a predicate, `FormalSymbolExtendedQ`. A formal symbol is reckoned "`Extended`" if it has correct syntax—a formal symbol followed by non-formals—and has the Protected attribute. It's possible to create syntactically correct but unprotected extended look-alikes, here's one: a42. It's not protected, so $\mathcal{UD}$ won't use it, and you shouldn't either.

A symbol can be a pure formal symbol, an extended formal symbol, or neither, but never both pure and extended.

We further implement a `SymbolInspector` for human monitoring and debugging of formal symbols. The creation of `SymbolInspector` ballooned into a full-blown, self-taught master class in evaluation control. The last section of this notebook presents our learning in detail for anyone who wants to wade in those waters.

The functions and tests in this notebook have been added to the Relativity Toolkit, and the rest of the toolit has been retrofitted to use such disciplined formals.

---

## Initialize the Relativity Toolkit

### Optional: Quit for a Fresh Kernel

- *Note, if you call* `Quit` *in a notebook, evaluation may hang. If you want to "Evaluate Notebook," mark the following two cells "Evaluatable" in the Cell→Property menu, call* `Quit` *and* `FrontEndTokenExecute` *once each, then comment out the cells, or mark them unevaluatable again. Alternatively, you can quit the kernel via the Evaluation menu (last item). At that point, you can "Evaluate Notebook" in a clean environment, or evaluate cells individually.*

```
Quit[];
```

In[•]:=
```
FrontEndTokenExecute["DeleteGeneratedCells"]
```

## Download the Code

In[1]:=
```
RTbranch = "v1.11.0";
RTbase = "https://raw.githubusercontent.com/rebcabin/RelativityToolkit/" <>
    RTbranch <> "/";
RTbust = "?t=" <> ToString[Round[AbsoluteTime[]]];(*Force fresh download*)
RTrules = RTbase <> "RelativityToolkit.wl" <> RTbust;
RTtests = RTbase <> "RelativityToolkit_RegressionTests.wl" <> RTbust;
RTviz = RTbase <> "RelativityToolkit_VisualShowcase.wl" <> RTbust;
Get[RTrules];
```

» Relativity Toolkit version : 1.11.0

» Relativity Connection Symbol : Γ

We must evaluate the regression test suite to load the test harness “Assert...” functions, which are exercised below in the body of this notebook.

In[8]:=
```
Get[RTtests];
```

```
================================================================
RUNNING REGRESSION SUITE v1.11.0
================================================================

--- SECTION 1: TYPE CHECKING (VALENCE) ---
[PASS]     1: Vector (x^μ) ⟶ {{μ}, {}}
[PASS]     2: Covector (p_ν) ⟶ {{}, {ν}}
[PASS]     3: Mixed Tensor (T^μ_ν) ⟶ {{μ}, {ν}}
[PASS]     4: Mixed Tensor (T_μν) ⟶ {{}, {μ, ν}}
[PASS]     5: Mixed Tensor (T^μν) ⟶ {{μ, ν}, {}}
[PASS]     6: Mixed Tensor (T^ν_μ) ⟶ {{ν}, {μ}}
[PASS]     7: Bare Symbol T ⟶ {{}, {}}
[PASS]     8: Bare Call T[] ⟶ {{}, {}}
[PASS]     9: Trap (Null) ⟶ {{}, {}}
[PASS]    10: Scalar Contraction (x^μ p_μ) ⟶ {{}, {}}
[PASS]    11: Scalar Contraction (x^μ p_μ) ⟶ {{}, {}}
[PASS]    12: Tensor-Vector (x^ν T^μ_ν → x^μ) ⟶ {{μ}, {}}
[PASS]    13: Tensor-Vector (x^μ T^μ_ν → x_ν) ⟶ {{}, {ν}}
[PASS]    14: Power of Scalar ⟶ {{}, {}}
```

[PASS] 15: Scalar Multiplication ⟶ {{$\mu$}, {}}

[PASS] 16: Gradient d(f)/dx$^{\alpha}$ -> Down[$\alpha$] ⟶ {{}, {$\alpha$}}

Valence Mismatch

» expect error message above {{}, {}}

[PASS] 17: Mismatch Handled Gracefully ⟶ {{}, {}}

--- SECTION 2: ALGEBRAIC CANONICALIZATION ---

[PASS] 18: Dummy Renaming ($A^{\mu}\ B_{\mu}$ ==$A^{\nu}\ B_{\nu}$) ⟶ $(p_{i1})\ (x^{i1})$

[PASS] 19: Commutativity ($A^{\mu}\ B_{\mu}$ == $B_{\nu}\ A^{\nu}$) ⟶ $(p_{i1})\ (x^{i1})$

[PASS] 20: Index Coalescing ($A^{\mu}\ B_{\mu}$ + $A^{\nu}\ B_{\nu}$ == 2 $A^{\rho}\ B_{\rho}$) ⟶ $2\ (p_{i1})\ (x^{i1})$

[PASS] 21: Double Dummy Pairs ($P^{\mu}\ S^{\nu}\ Q_{\mu}\ T_{\nu}$) ⟶ $(P^{i1})\ (Q_{i1})\ (S^{i2})\ (T_{i2})$

[PASS] 22: Free Indices Preserved ⟶ $\{(A^{\alpha})\ (B^{\beta}), (A^{i1})\ (B^{i1})\}$

--- SECTION 3: PHYSICS, METRIC & TRANSFORMATIONS ---

[PASS] 23: Lowering Index ($A^{\nu}\ g_{\mu\nu}$ → $A_{\mu}$) ⟶ $A_{\mu}$

[PASS] 24: Raising Index ($g^{\mu\nu}\ p_{\nu}$ → $p^{\mu}$) ⟶ $p^{\mu}$

[PASS] 25: Inverse Metric ($g^{\mu\alpha}\ g_{\alpha\nu}$ → $\delta^{\mu}_{\nu}$) ⟶ $\delta^{\mu}_{\nu}$

[PASS] 26: Alpha-Conversion: distinct indices) ⟶ $\{\mu 11, \mu 12\}$

[PASS] 27: Triple Metric Sandwich (g.g.g -> g) ⟶ $g_{\mu\ \nu}$

[PASS] 28: Delta-Gamma Contraction ⟶ $\Gamma^{s}_{a\,b}$

=== Metric Differentiation Rules ===

[PASS] 29: Permissive Contraction ⟶ $\delta^{nu}_{lam}$

[PASS] 30: Metric Symmetry in Derivatives ⟶ 0

[PASS] 31: Delta Chain Rule ⟶ $f[x]_{,\mu}$

[PASS] 32: Metric Compatibility ⟶ 0

[PASS] 33: Levi-Civita Index Safety (Unique indices generated) ⟶ {$\sigma$17, $\sigma$18}

--- SECTION 4: METRIC EQUIVALENCE ($A^{\mu}\ B_{\mu}$ == $B^{\nu}\ A_{\nu}$) ---

[PASS] 34: Structural Distinction ($A^{\mu}\ B_{\mu}$ ≠ $B^{\nu}\ A_{\nu}$) initially
⟶ $\{(A^{i1})\ (B_{i1}), (A_{i1})\ (B^{i1})\}$

[PASS] 35: Metric Equivalence ($A_{\nu}\ B^{\nu}$ reduces to $A^{\mu}\ B_{\mu}$) ⟶ $(A^{i1})\ (B_{i1})$

--- SECTION 5: SECOND DERIVATIVE BOX STRUCTURE ---

[PASS] 36: Found FractionBox + ^2 ⟶ True

--- SECTION 6: CALCULUS ENGINE ---

[PASS] 37: Leibniz Product Rule ⟶ $g\ f_{,\mu} + f\ g_{,\mu}$

[PASS] 38: Covariant Derivative Expansion ⟶ True

[PASS] 39: Power Rule ⟶ $2\left(a\$10035^{i1}\right)\left(a\$10035^{i1}{}_{,\nu}\right)$

[PASS] 40: Differentiation Linearity ⟶ True

[PASS] 41: CD of Scalar reduces to Partial ⟶ $phi\$10048_{,\mu}$

[PASS] 42: Scalar CD contains no Connection Coefficients ⟶ True

[PASS] 43: Nested CD Alpha-Conversion (Unique Indices Generated) ⟶ True

--- SECTION 7: QUOTIENT THEOREM (Extraction) ---

[PASS] 44: Quotient Extraction (Simple) ⟶ $T_{s}$

[PASS] 45: Quotient Extraction (Distributed Dummies) ⟶ $R_{s}+S_{s}$

[PASS] 46: Quotient Extraction (Zero Case) ⟶ 0

--- SECTION 8: TENSOR FORM ROBUSTNESS ---

[PASS] 47: TensorForm maps arbitrary formals (Q, Z, I99) to Greek ⟶ True

--- SECTION 9: COMMA NOTATION DISPLAY LOGIC ---

[PASS] 48: Atomic Tensor ⟶ $A^{\mu}{}_{,\nu}$

[PASS] 49: Sealed Product (Gamma Glue) ⟶ $\left(\left(A^{\mu}\right)\Gamma^{a}_{b\,c}\right)_{,\nu}$

[PASS] 50: Sealed Sum ⟶ $(A+B)_{,\nu}$

[PASS] 51: Sealed Power ⟶ $\left(A^{2}\right)_{,\nu}$

[PASS] 52: Arbitrary Function (Sin) ⟶ $Sin[theta]_{,\nu}$

[PASS] 53: Nested Math (A*(B+C)) ⟶ $(A\,(B+C))_{,\nu}$

--- SECTION 10: EINSTEIN SUMMATION (Contract & ContractAll) ---

[PASS] 54: Single Contraction (A^u B_u) ⟶ $2\left(A^{i1}\right)\left(B_{i1}\right)$

[PASS] 55: Contract (One-Shot) expanded exactly one pair ⟶ {False, True}

[PASS] 56: ContractAll expanded all pairs recursively ⟶ True

[PASS] 57: Variance Handling (Divergence: d(A^u)/dx^u) ⟶ $2\left(A^{i1}{}_{,i1}\right)$

[PASS] 58: Distribution over Sum (A^u B_u + C^v D_v) ⟶ True

[PASS] 59: Free Indices Protected (No Contraction) ⟶ $\left(A^{\mu}\right)\left(B_{\nu}\right)$

[PASS] 60: Coordinates Excluded from Scanner ⟶ $A^{1}$

--- SECTION 11: ELECTRODYNAMICS & GAUGE INVARIANCE ---

[PASS] 61: U(1) Gauge Covariance $\left(\left(\psi\ D_{\mu}\right)'==e^{i e\Lambda}\ \psi\ D_{\mu}\right)$ ⟶

$$-i\,e^{i\,e\$10152\,\Lambda\$10152}\,e\$10152\,\psi\$10152\,\left(A\$10152_{\ \mu}\right)+e^{i\,e\$10152\,\Lambda\$10152}\,\psi\$10152_{,\mu}$$

[PASS] 62: Field Strength Invariance (F' == F) ⟶ $-\left(A\$10152_{\ m,n}\right)+A\$10152_{\ n,m}$

[PASS] 63: Weinberg-Salam Mixed Covariance (Geometric + Gauge)
$\longrightarrow e^{i\, gW\$10167\, \Lambda\$10167}\, (W\$10167\ {}^{\alpha}{}_{,\mu}) - i\, e^{i\, gW\$10167\, \Lambda\$10167}\, gW\$10167\, (W\$10167\ {}^{\alpha})\, (Z\$10167\ {}_{\mu}) + e^{i\, gW\$10167\, \Lambda\$10167}\, \left(W\$10167\ {}^{\dot{j}1}\right)\, \Gamma^{\alpha}_{\mu\, \dot{j}1}$

--- SECTION 12: RIEMANN CURVATURE DERIVATION ---

[PASS] 64: Riemann Derivation matches Textbook $\longrightarrow \Gamma^{\mu}_{\dot{s}\,\nu,\rho} - \Gamma^{\mu}_{\dot{s}\,\rho,\nu} - \Gamma^{\dot{j}1}_{\dot{s}\,\rho}\, \Gamma^{\mu}_{\dot{j}1\,\nu} + \Gamma^{\dot{j}1}_{\dot{s}\,\nu}\, \Gamma^{\mu}_{\dot{j}1\,\rho}$

--- SECTION 13: GEOMETRY OPERATORS (Gradient, Laplacian) ---

[PASS] 65: Gamma Factory (Spherical $\Gamma_{r\theta\theta}$) $\longrightarrow$ -r

[PASS] 66: GradRaised (Radial Function $r^2$) $\longrightarrow$ {0, 2 r, 0, 0}

[PASS] 67: GradSquared (Null Vector Norm) $\longrightarrow$ 0

[PASS] 68: ScalarLaplacian (Harmonic 1/r) $\longrightarrow$ 0

[PASS] 69: ScalarLaplacian (Polynomial $r^2$) $\longrightarrow$ 6

--- SECTION 14: FORMAL SYMBOLS ---

--- (1) Identity & Built-in Protection ---

[PASS] 70: input: formal symbol literal $\longrightarrow$ Pure

[PASS] 71: input: formal symbol string $\longrightarrow$ Pure

[PASS] 72: input: extended symbol literal $\longrightarrow$ Extended

[PASS] 73: input: extended symbol string $\longrightarrow$ Extended

[PASS] 74: non-formal symbol literal $\longrightarrow$ Neither

[PASS] 75: non-formal string $\longrightarrow$ Neither

[PASS] 76: non-formal empty string $\longrightarrow$ Neither

[PASS] 77: empty string $\longrightarrow$ $Failed

[PASS] 78: integer $\longrightarrow$ $Failed

[PASS] 79: Identity: Built-in $\dot{j}$ is Protected $\longrightarrow$ True

[PASS] 80: Identity: Latin Pure Formal recognized $\longrightarrow$ True

[PASS] 81: Identity: Greek Pure Formal recognized $\longrightarrow$ True

--- (2) Pointer & Construction Logic ---

[PASS] 82: Pointer: Variable holding symbol recognized $\longrightarrow$ True

[PASS] 83: Pointer: Target Extended symbol $\dot{j}888$ is Protected $\longrightarrow$ True

[PASS] 84: Constructor: Rejects empty string $\longrightarrow$ True

--- (3) Structural Variety & Boundaries ---

[PASS] 85: Structure: Numbered dummy recognized $\longrightarrow$ True

[PASS] 86: Structure: Prefixed symbol rejected $\longrightarrow$ True

[PASS] 87: Boundary: Pure is NOT Extended $\longrightarrow$ True

[PASS] 88: Boundary: Extended is NOT Pure $\longrightarrow$ True

```
[PASS]    89: Boundary: Ordinary Symbol is not Pure System formal → True
[PASS]    90: Boundary: String is not Pure System formal symbol → True
[PASS]    91: Boundary: Number is not Pure System formal symbol → True
[PASS]    92: Boundary: Ordinary Symbol is not Extended formal → True
[PASS]    93: Boundary: String is not Extended formal symbol → True
[PASS]    94: Boundary: Number is not Extended formal symbol → True
--- (4) Scoping Integrity (The 'Hold' Check) ---
[PASS]    95: Scoping: Built-in identity persists despite value 5 → True
[PASS]    96: Non-system symbol not protected, thus not extended. → True
[PASS]    97: Scoping: Extended identity after explicit protection → True
--- (5) Constructor Validation ---
[PASS]    98: Safety: Reject double formal prefix → True
[PASS]    99: Safety: Reject plain string → True
[PASS]   100: Structure: Accept single formal prefix → True
--- (6) Start-Character Enforcement ---
[PASS]   101: Strictness: Reject buried formal (ExtendedQ -> False) → True
[PASS]   102: Strictness: Reject buried formal (PureQ -> False) → True
[PASS]   103: Strictness: Reject suffix formal (ExtendedQ -> False) → True
[PASS]   104: Strictness: Reject suffix formal (PureQ -> False) → True
--- (6) Polymorphism ---
[PASS]   105: Polymorphism: String "i" is a Pure symbol → True
[PASS]   106: Polymorphism: String "i999" is an Extended symbol → True

------------------------------------------------------------------
REGRESSION SUITE COMPLETE
PASSED: 106
FAILED: 0
STATUS: GREEN (Ready for Publication)
------------------------------------------------------------------
```

# Formal Symbol Character Ranges

The following list is simply scraped from *the Wolfram documentation*.

In[9]:=
```
allFormals = {
a, α, b, β, c, A, A, B, B, C, X, D, Δ, F, E, E,
H, F, G, Γ, H, I, I, J, K, K, Φ, L, Λ, M, M, N,
N, O, Ω, O, P, Φ, Π, Ψ, Q, R, R, S, ∂, Σ, ς, T,
T, Θ, U, ϒ, V, W, X, Ξ, Y, Z, Z, χ, ϒ, ε, ϰ, φ,
ϖ, ϱ, ϑ, d, δ, ϝ, e, ϵ, η, f, ς, g, ɣ, h, i, ι,
j, k, ϰ, ο, l, λ, m, μ, n, ν, o, ω, ο, p, ϕ, π,
ψ, q, r, ρ, s, ϶, a, b, c, A, B, C, D, E, F, G,
H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V,
W, X, Y, Z, d, e, f, g, h, i, j, k, ℓ, m, n, o,
p, q, r, s, t, u, v, w, x, y, z, σ, ς, t, τ, Θ, u,
υ, v, w, x, ξ, y, z, ξ};
```

There are 168 of them:

In[10]:=
```
Length[allFormals]
```

Out[10]=
```
168
```

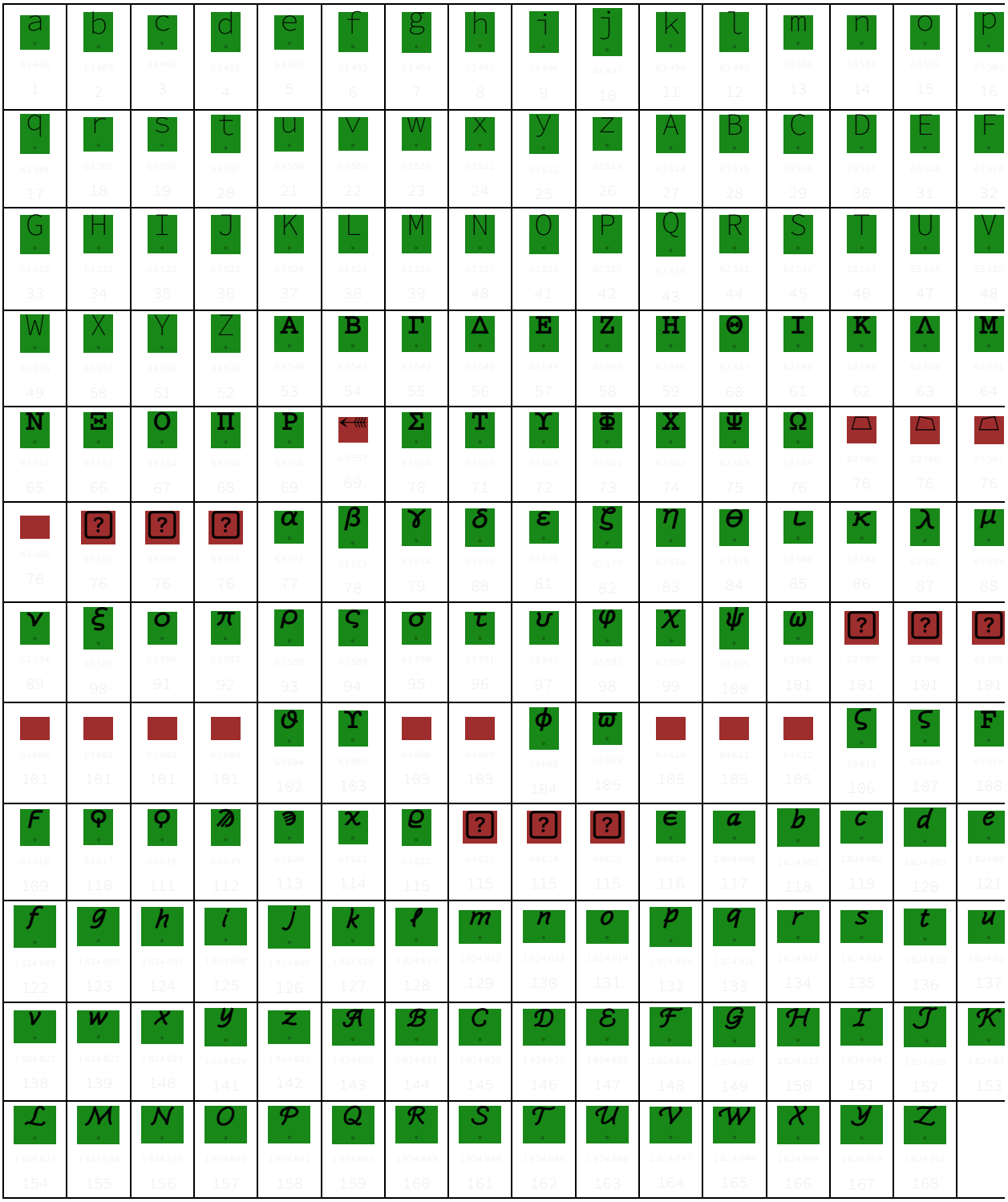
They live in a few ranges of character codes high in the Unicode sphere. **showCodes**, below, generates a visual map of the character codes used by Wolfram for formal symbols.

- Green Tiles: Valid, assigned formal symbols (e.g., **a**, **α**).
- Red Tiles: Unassigned or invalid slots (gaps in the sequence).

The visualization iterates through the integer ranges and queries **formalCharCodeQ** for every number to decide whether it should be colored Green (Valid) or Red (Invalid).

In[11]:=
```
ClearAll[formalCharCodeQ];
formalCharCodeQ[code_Integer] := (
    (63 488 ≤ code ≤ 63 556) || (*Latin and primary math*)
     (63 558 ≤ code ≤ 63 564) || (*Remaining math symbols after arrow*)
     (63 572 ≤ code ≤ 63 596) || (*Lowercase Greek*)
     (63 604 ≤ code ≤ 63 605) || (*Mathematical variants*)
     (63 608 ≤ code ≤ 63 609) || (*Greek variants*)
     (63 613 ≤ code ≤ 63 622) || (code === 63 626) ||
     (*Final Greek variants & oddities*)(1 024 000 ≤ code ≤ 1 024 051));
ClearAll[showCodes];
showCodes[] :=
  Module[{codes = Range[63 488, 63 626] ~ Join ~ Range[1 024 000, 1 024 051], gcount = 0},
   Grid[Partition[MapIndexed[Column[{
          Item[Style[Symbol@FromCharacterCode@#1, 20, Bold],
           Background → If[formalCharCodeQ[#1],
              (gcount++;
               Darker[StandardGreen]),
              Darker[StandardRed]]],
          Style[#1, 6, GrayLevel[0.90]],
          Style[gcount, 10, GrayLevel[0.90]]},
         Alignment → Center] &, codes],
     UpTo[16]],
    Frame → All, ItemSize → {2.5, 2.5}, Background → {None, None, {}}]];
showCodes[]
```

Out[15]=



# Check Formal Identity

**checkFormalIdentity** is a low-level, polymorphic interrogator of the structure of an input **Symbol** or

`String`. It does not evaluate its input. It returns one of **"Pure"**, **"Extended"**, or **"Neither"**. It fails loudly (with `$Failed`) on any other type of input.

`checkFormalIdentity` has a two-layer defense against evaluation, critical when inspecting symbols that might have values assigned.

1. `SetAttributes[..., HoldFirst]`: stops the argument `s` from evaluating before it enters the function. Without this, calling the function on a variable `x` would pass the value of `x` instead of `x`.
2. `Unevaluated[...]`: Even inside a `HoldFirst` function, passing `s` to certain other functions can trigger evaluation. Wrapping occurrences of `s` in `Unevaluated`, e.g., `SymbolName[Unevaluated[s]]`, guarantees that the called function operates on a symbol itself, not its value. `Unevaluated` has no effect on a `String`.

In[16]:=
```
ClearAll[checkFormalIdentity];
SetAttributes[checkFormalIdentity, HoldFirst];
checkFormalIdentity[s : (_Symbol | _String)] :=
  Module[{name, codes},
   name = Which[
     Head[Unevaluated[s]] === Symbol, SymbolName[Unevaluated[s]],
     (* if input is a literal string, not a symbol *)
     StringQ[Unevaluated[s]], s,
     True, Return[$Failed]];
   codes = ToCharacterCode[name];
   Which[
    (Length[codes] == 1 && formalCharCodeQ[First[codes]]), "Pure",
    (Length[codes] > 0 && formalCharCodeQ[First[codes]]), "Extended",
    True, "Neither"
   ]];
checkFormalIdentity[___] := $Failed;
```

Unit tests appear below.

# Formal Symbol Predicate

Except for the `HoldAll` attribute, this implementation is obvious, given `checkFormalIdentity`.

In[20]:=
```
ClearAll[FormalSymbolQ];
SetAttributes[{FormalSymbolQ}, HoldAll];
FormalSymbolQ[s_] := "Pure" === checkFormalIdentity[s];
```

# Extended Formal Symbol Predicate

One must write `FormalSymbolExtendedQ[Evaluate[x]]` to check the value of an expression x, even if that expression is just a symbol whose value is a constructed extended formal symbol. Many examples and tests below should make this clear.

In[23]:=
```
ClearAll[FormalSymbolExtendedQ];
SetAttributes[{FormalSymbolExtendedQ}, HoldAll];
FormalSymbolExtendedQ[s_] :=
  checkFormalIdentity[s] === "Extended" && MemberQ[Attributes[s], Protected];
```

# Constructor for Extended Formal Symbols

`CreateExtendedFormal` creates a formal symbol, extended if there are non-formals after a leading formal, pure if there is only a single formal. I.e., it has no effect if given a pure formal.

In[26]:=
```
ClearAll[CreateExtendedFormal];
CreateExtendedFormal::invalidStructure =
  "The name '`1`' is invalid. Extended formals must start with
    exactly one formal character followed by non-formal characters.";
Module[{valid},
  valid[codes_] := Length[codes] > 0 &&
    formalCharCodeQ[First[codes]] &&
    ! AnyTrue[Rest[codes], formalCharCodeQ];
  CreateExtendedFormal[name_String] :=
   If[valid[ToCharacterCode[name]],
    With[{s = Symbol[name]},
     Protect[s]; s],
    (Message[CreateExtendedFormal::invalidStructure, name];
     Return[$Failed])];
  CreateExtendedFormal[sym_Symbol] :=
   CreateExtendedFormal[SymbolName[sym]]; ];
```

# Test Suite

This has been added to the regression suite of the Relativity Toolkit.

In[29]:=
```
(*SETUP*)
Protect[i123];
protectedDummy = CreateExtendedFormal["j888"];
CreateExtendedFormal[i999];

Print["--- (1) Identity & Built-in Protection ---"];

(*Case 0: low-level checks*)
AssertEqual[checkFormalIdentity[i],
  "Pure", "0.1. input: formal symbol literal"];
AssertEqual[checkFormalIdentity["i"], "Pure", "0.2. input: formal symbol string"];
```

```
AssertEqual[checkFormalIdentity[j1234],
  "Extended", "0.3. input: extended symbol literal"];
AssertEqual[checkFormalIdentity["j2345"],
  "Extended", "0.4. input: extended symbol string"];
AssertEqual[checkFormalIdentity[foo],
  "Neither", "0.5. non-formal symbol literal"];
AssertEqual[checkFormalIdentity["foo"], "Neither", "0.6. non-formal string"];
AssertEqual[checkFormalIdentity[""], "Neither", "0.7. non-formal empty string"];
AssertEqual[checkFormalIdentity[], $Failed, "0.8. empty string"];
AssertEqual[checkFormalIdentity[42], $Failed, "0.9. integer"];

(*Case 1: System Protection*)
AssertProtected[i, "1. Identity: Built-in i is Protected"];

(*Case 2: Latin Identification*)
AssertTrue[FormalSymbolQ[i], "2. Identity: Latin Pure Formal recognized"];

(*Case 3: Greek Identification*)
AssertTrue[FormalSymbolQ[α], "3. Identity: Greek Pure Formal recognized"];

Print["--- (2) Pointer & Construction Logic ---"];

(*Case 4: Pointer Resolution (MUST EVALUATE THE POINTER)*)
AssertTrue[FormalSymbolExtendedQ[Evaluate@protectedDummy],
  "4. Pointer: Variable holding symbol recognized"];

(*Case 5: Target is Protected*)
AssertProtected[protectedDummy, "5. Pointer: Target symbol j888 is Protected"];

(*Case 6: Negative Constructor Check*)
AssertMatchQ[Quiet[CreateExtendedFormal[""]], $Failed,
  "6. Constructor: Rejects empty string"];

Print["--- (3) Structural Variety & Boundaries ---"];

(*Case 7: Numbered Dummies (Protected it in Setup)*)
AssertTrue[FormalSymbolExtendedQ[i123],
  "7. Structure: Numbered dummy recognized"];

(*Case 8: Reject Prefixed Formal Symbols*)
AssertFalse[FormalSymbolExtendedQ[Partialx],
  "8. Structure: Prefixed symbol rejected"];
```

```
(*Case 9: Pure System is NOT Extended*)
AssertFalse[FormalSymbolExtendedQ[i], "9. Boundary: Pure is NOT Extended"];

(*Case 10: Extended is NOT Pure System*)
AssertFalse[FormalSymbolQ[i123], "10. Boundary: Extended is NOT Pure"];

AssertFalse[FormalSymbolQ[foo],
  "10.a. Boundary: Ordinary Symbol is not Pure System formal"];
AssertFalse[FormalSymbolQ["foo"],
  "10.b. Boundary: String is not Pure System formal symbol"];
AssertFalse[FormalSymbolQ[42],
  "10.c. Boundary: Number is not Pure System formal symbol"];

AssertFalse[FormalSymbolExtendedQ[foo],
  "10.d. Boundary: Ordinary Symbol is not Extended formal"];
AssertFalse[FormalSymbolExtendedQ["foo"],
  "10.e. Boundary: String is not Extended formal symbol"];
AssertFalse[FormalSymbolExtendedQ[42],
  "10.f. Boundary: Number is not Extended formal symbol"];

Print["--- (4) Scoping Integrity (The 'Hold' Check) ---"];

Block[{x = 10, i = 5, i123 = 7},

  (*Case 11: System Symbol Value-Shielding*)
  AssertTrue[FormalSymbolQ[i],
   "11. Scoping: Built-in identity persists despite value 5"];

  (*Case 12: Extended Symbol (non-System) is not protected *)
  AssertFalse[FormalSymbolExtendedQ[i123],
   "12a. Non-system symbol not protected, thus not extended."];

  Protect[i123];
  AssertTrue[FormalSymbolExtendedQ[i123],
   "12b. Scoping: Extended identity after explicit protection"]
 ];

Print["--- (5) Constructor Validation ---"];

(*Case 13: Reject double formal prefix (e.g.,kj...)*)
AssertMatchQ[Quiet[CreateExtendedFormal["kj888"]],
```

```
  $Failed, "13. Safety: Reject double formal prefix"];

(*Case 14: Reject plain string without any formal symbol*)
AssertMatchQ[Quiet[CreateExtendedFormal["plainVar"]],
  $Failed, "14. Safety: Reject plain string"];

(*Case 15: Accept valid single formal prefix*)
AssertTrue[Head[CreateExtendedFormal["k888"]] === Symbol,
  "15. Structure: Accept single formal prefix"];

Print["--- (6) Start-Character Enforcement ---"];

(*Case 16: Reject'Buried' Formal Symbol (e.g.vari123)*)
AssertFalse[FormalSymbolExtendedQ["vari123"],
  "16. Strictness: Reject buried formal (ExtendedQ -> False)"];

AssertFalse[FormalSymbolQ["vari123"],
  "17. Strictness: Reject buried formal (PureQ -> False)"];

(*Case 18: Reject 'Suffix' Extended Symbol (e.g.xi)*)
AssertFalse[FormalSymbolExtendedQ["xi"],
  "18. Strictness: Reject suffix formal (ExtendedQ -> False)"];

(*Case 18: Reject'Suffix' Pure Symbol (e.g.xi)*)
AssertFalse[FormalSymbolQ["xi"],
  "19. Strictness: Reject suffix formal (PureQ -> False)"];

Print["--- (6) Polymorphism ---"]

(*True: The string "i" describes a valid Pure symbol.*)
AssertTrue[FormalSymbolQ["i"],
  "20. Polymorphism: String \"i\" is a Pure symbol"];

(*True: The string "i" describes a valid Extended symbol.*)
AssertTrue[FormalSymbolExtendedQ["i999"],
  "21. Polymorphism: String \"i999\" is an Extended symbol"];

Print["--- TEST SUITE COMPLETE: 100% COVERAGE ---"];
--- (1) Identity & Built-in Protection ---
[PASS]   107: 0.1. input: formal symbol literal ⟶ Pure
[PASS]   108: 0.2. input: formal symbol string ⟶ Pure
```

```
[PASS]   109: 0.3. input: extended symbol literal ⟶ Extended
[PASS]   110: 0.4. input: extended symbol string ⟶ Extended
[PASS]   111: 0.5. non-formal symbol literal ⟶ Neither
[PASS]   112: 0.6. non-formal string ⟶ Neither
[PASS]   113: 0.7. non-formal empty string ⟶ Neither
[PASS]   114: 0.8. empty string ⟶ $Failed
[PASS]   115: 0.9. integer ⟶ $Failed
[PASS]   116: 1. Identity: Built-in i is Protected ⟶ True
[PASS]   117: 2. Identity: Latin Pure Formal recognized ⟶ True
[PASS]   118: 3. Identity: Greek Pure Formal recognized ⟶ True
--- (2) Pointer & Construction Logic ---
[PASS]   119: 4. Pointer: Variable holding symbol recognized ⟶ True
[PASS]   120: 5. Pointer: Target symbol j888 is Protected ⟶ True
[PASS]   121: 6. Constructor: Rejects empty string ⟶ True
--- (3) Structural Variety & Boundaries ---
[PASS]   122: 7. Structure: Numbered dummy recognized ⟶ True
[PASS]   123: 8. Structure: Prefixed symbol rejected ⟶ True
[PASS]   124: 9. Boundary: Pure is NOT Extended ⟶ True
[PASS]   125: 10. Boundary: Extended is NOT Pure ⟶ True
[PASS]   126: 10.a. Boundary: Ordinary Symbol is not Pure System formal ⟶ True
[PASS]   127: 10.b. Boundary: String is not Pure System formal symbol ⟶ True
[PASS]   128: 10.c. Boundary: Number is not Pure System formal symbol ⟶ True
[PASS]   129: 10.d. Boundary: Ordinary Symbol is not Extended formal ⟶ True
[PASS]   130: 10.e. Boundary: String is not Extended formal symbol ⟶ True
[PASS]   131: 10.f. Boundary: Number is not Extended formal symbol ⟶ True
--- (4) Scoping Integrity (The 'Hold' Check) ---
[PASS]   132: 11. Scoping: Built-in identity persists despite value 5 ⟶ True
[PASS]   133: 12a. Non-system symbol not protected, thus not extended. ⟶ True
[PASS]   134: 12b. Scoping: Extended identity after explicit protection ⟶ True
--- (5) Constructor Validation ---
[PASS]   135: 13. Safety: Reject double formal prefix ⟶ True
[PASS]   136: 14. Safety: Reject plain string ⟶ True
[PASS]   137: 15. Structure: Accept single formal prefix ⟶ True
--- (6) Start-Character Enforcement ---
[PASS]   138: 16. Strictness: Reject buried formal (ExtendedQ -> False) ⟶ True
[PASS]   139: 17. Strictness: Reject buried formal (PureQ -> False) ⟶ True
```

```
[PASS]   140: 18. Strictness: Reject suffix formal (ExtendedQ -> False) ⟶ True
[PASS]   141: 19. Strictness: Reject suffix formal (PureQ -> False) ⟶ True
--- (6) Polymorphism ---
[PASS]   142: 20. Polymorphism: String "i" is a Pure symbol ⟶ True
[PASS]   143: 21. Polymorphism: String "i999" is an Extended symbol ⟶ True
--- TEST SUITE COMPLETE: 100% COVERAGE ---
```

# Symbol Inspector

## Design & internal Logic

### Purpose

For displaying and debugging formal symbols, including our extended formal symbols, **SymbolInspector** exposes the internal properties of any symbol, indirecting when necessary when its input evaluates to the formal symbol we want, metaphorically treating the input as a *pointer*.

### Usage

**SymbolInspector[symbolLiteral]**
**(* or *)**
**SymbolInspector[Evaluate[expressionWithValue]]**

- On a Literal: **SymbolInspector[x]** inspects **x**.
- On an Expression: e.g., **SymbolInspector[var]**, evaluates the expression **var** first, then inspects the resulting value, which may be the value we want.

### Examples

Inspect a formal symbol input as a literal:

**SymbolInspector[i]**

Out[ ]=

| Property | Value |
|---|---|
| Literal Symbol | i |
| Own Value | None (Undefined) |
| Points To | N/A (Literal) |
| Context | System` |
| Character Codes | {63 496} |
| Classification | Pure System |
| Attributes | {Protected} |
| Definition Type | No Definitions |

Indirect a *metaphorical pointer*: Evaluate a symbol to get the formal symbol we *Really* want to inspect

In[•]:=
```
Module[{x = i}, SymbolInspector[x]]
```

Out[•]=

| Property | Value |
|---|---|
| Literal Symbol | x$15190 |
| Own Value | i |
| Points To | i |
| Context | System` |
| Character Codes | {63 496} |
| Classification | Pure System |
| Attributes | {Protected} |
| Definition Type | OwnValue (Variable) |

## Mechanics

**SymbolInspector** relies on three metaprogramming features:

1. **SetAttributes[..., HoldAll]**

   **Problem**: Normally, if $x = 5$, typing **f[x]** evaluates to **f[5]**: **x** is evaluated before being "sent" into **f**.

   **Solution**: **HoldAll** prevents **x** from evaluating, lettng the raw symbol **x** through for deeper inspection.

2. **OwnValues** Introspection

   **Problem**: In Mathematica, variables normally store definitions (if any) in **OwnValues**, whereas functions (**f[x_] := x**) store definitions in **DownValues**.

   **Solution**: To detect whether **x** is a pointer to another symbol, interrogate **OwnValues[x]**. If we find a rule like **{x :> y}**, we know **x** is a pointer.

3. "Safe Peek" Pattern

   Code: **ReleaseHold[Replace[Hold[s], sym_Symbol :> OwnValues[sym], {1}]]**

   This specific idiom extracts a symbol's **OwnValue** without letting the symbol evaluate.

   **3.1.** **Hold[s]** freezes the symbol.

   **3.2.** **Replace** swaps the symbol for its **OwnValues** rule list inside the hold.

   **3.3.** **ReleaseHold** lets us look at the rule list.

## Learning

Simply calling **OwnValues[s]** inside a held function often evaluates **s** too early. The idiom above was the only way we found to get the definition safely.

## "Smart Pointer" Logic

The distinctive feature of this tool is the **isPointer** check.

Logic: **MatchQ[eval, {(_ :> _Symbol)}]**

Behavior:

1. If **x** is defined as **x = y** where **y** is a symbol, the Inspector detects this pattern.

**2.** It “dereferences” the pointer, switching focus to `y`, ensuring that inspecting a variable holding a formal symbol yields the properties of the formal symbol content, not the properties of the “pointer” variable (the container).

In[75]:=
```
ClearAll[SymbolInspector];
SetAttributes[SymbolInspector, HoldAll];
SymbolInspector[s_Symbol] :=
  Module[{ownVals, isPointer, heldSym},
   (*1. Peek inside the pointer safely*)
   ownVals = OwnValues[Unevaluated[s]];
   isPointer = MatchQ[ownVals, {(_ :> _Symbol)}];
   heldSym = If[isPointer,
     Hold[Evaluate[ownVals[[1, 2]]]],
     Hold[s]];
   (*2. Use 'With' to 'lock in' the symbol we are actually inspecting*)
   Replace[heldSym, Hold[sym_] :>
     With[{
       name = SymbolName[Unevaluated[sym]],
       attr = Attributes[Unevaluated[sym]],
       ctx = Context[Unevaluated[sym]]},
      Grid[{
        {"Property", "Value"},
        {"Literal Symbol", HoldForm[s]},
        {"Own Value", If[ownVals === {}, "None (Undefined)", s]},
        {"Points To", If[isPointer, sym, "N/A (Literal)"]},
        {"Context", ctx},
        {"Character Codes", ToCharacterCode[name]},
        {"Classification", Which[
          (* don't forget to Evaluate! *)
          FormalSymbolQ[Unevaluated[sym]], "Pure System",
          FormalSymbolExtendedQ[Unevaluated[sym]], "Extended",
          True, "User"]},
        {"Attributes", attr},
        {"Definition Type", Which[
          ownVals =!= {}, "OwnValue (Variable)",
          DownValues[Unevaluated[s]] =!= {}, "DownValue (Function)",
          True, "No Definitions"]}},
       Frame -> All,
       Alignment -> Left,
       Background -> {None, {GrayLevel[0.3], Black}}]]]];
SymbolInspector[___] := $Failed;
```

## Case Analysis

We can pass in

- naked symbols, naked strings,
- naked symbols and strings in variables
- construction expressions for formal symbols (with Evaluate!),
- constructed formal symbols from symbols or string in variables.

Considering both **Pure** (non-extended) and **Extended** symbols, that makes 32 cases, plus a few corner cases. We don't cover all the cases in the examples below, but we do point out some recommended usages and some delicate points.

### Naked Symbols and Strings

DO THIS: Note the System namespace; Pure formals live in **System`**. Also note that the symbol is automatically protected by Wolfram and has no **OwnValue**.

In[79]:=
```
SymbolInspector[i]
```

Out[79]=

| Property | Value |
|---|---|
| Own Value | None (Undefined) |
| Points To | N/A (Literal) |
| Context | System` |
| Character Codes | {63 496} |
| Classification | Pure System |
| Attributes | {Protected} |
| Definition Type | No Definitions |

DON'T DO THIS: Expect loud failure on strings.

In[80]:=
```
SymbolInspector[".i"]
```

Out[80]=

```
$Failed
```

DON'T DO THIS: You can't assign to a pure system formal, by Wolfram design:

In[81]:=
```
z = 42;
```

Set: Symbol z is Protected.

DON'T DO THIS: A Module will let you have the illusion of assigning a value to a formal symbol, but the symbol won't be any kind of formal. It's a **User** symbol rather than **Pure Formal** or **Extended**, the only alternatives to **User**. The character codes are long because the symbols "real" name is i followed by some random-looking numbers (actually internal counters and dollar signs and what not).

In[82]:=
```
Module[{i = 5}, SymbolInspector[i]]
```

Out[82]=

| Property | Value |
|---|---|
| Own Value | 5 |
| Points To | N/A (Literal) |
| Context | System` |
| Character Codes | {63 496, 36, 49, 48, 52, 51, 56} |
| Classification | User |
| Attributes | {Temporary} |
| Definition Type | OwnValue (Variable) |

DON'T DO THIS: You CAN assign values to things that look like naked extended formals, but it is highly discouraged. They're not extended. They appear in the **User** classification. The only real extended formals are constructed ones (examples below). **There is no such thing as a naked, literal extended formal**. The constructor will protect constructed extended formals, as we see below, so you can't assign values to them after-the-fact.

In[83]:=
```
zDontDoThis = 42;
SymbolInspector[zDontDoThis]
```

Out[84]=

| Property | Value |
|---|---|
| Own Value | 42 |
| Points To | N/A (Literal) |
| Context | Global` |
| Character Codes | {63 513, 68, 111, 110, 116, 68, 111, 84, 104, 105, 115} |
| Classification | User |
| Attributes | {} |
| Definition Type | OwnValue (Variable) |

DON'T DO THIS: Even without values, things that look like literal extended symbols just are not.

In[85]:=
```
SymbolInspector[z888]
```

Out[85]=

| Property | Value |
|---|---|
| Own Value | None (Undefined) |
| Points To | N/A (Literal) |
| Context | Global` |
| Character Codes | {63 513, 56, 56, 56} |
| Classification | User |
| Attributes | {} |
| Definition Type | No Definitions |

## Naked Symbols in Variables

DO THIS: Everything above works as expected, just through variables acting, metaphorically, as

pointers.

In[86]:=
```
Module[{var = i}, SymbolInspector[var]]
```

Out[86]=

| Property | Value |
|---|---|
| Own Value | i |
| Points To | i |
| Context | System` |
| Character Codes | {63 496} |
| Classification | Pure System |
| Attributes | {Protected} |
| Definition Type | OwnValue (Variable) |

## Directly Constructed Symbols

DO THIS: Here is the way to create an extended formal without assigning it to a variable. To inspect it with such direct construction, you must precede the call of **CreateExtendedFormal** with an **Evaluate**.

In[87]:=
```
SymbolInspector[Evaluate@CreateExtendedFormal[k123]]
```

Out[87]=

| Property | Value |
|---|---|
| Own Value | None (Undefined) |
| Points To | N/A (Literal) |
| Context | Global` |
| Character Codes | {63 498, 49, 50, 51} |
| Classification | Extended |
| Attributes | {Protected} |
| Definition Type | No Definitions |

DO THIS: After you do this, the symbol itself, as a literal, now has properties that mark it as a bona-fide extended formal:

In[88]:=
```
SymbolInspector[k123]
```

Out[88]=

| Property | Value |
|---|---|
| Own Value | None (Undefined) |
| Points To | N/A (Literal) |
| Context | Global` |
| Character Codes | {63 498, 49, 50, 51} |
| Classification | Extended |
| Attributes | {Protected} |
| Definition Type | No Definitions |

DON'T DO THIS: Recall that if it has not been constructed, it's not an extended formal:

In[89]:=
```
SymbolInspector[k456]
```

Out[89]=

| Property | Value |
|---|---|
| Own Value | None (Undefined) |
| Points To | N/A (Literal) |
| Context | Global` |
| Character Codes | {63 498, 52, 53, 54} |
| Classification | User |
| Attributes | {} |
| Definition Type | No Definitions |

DO THIS: If you construct an extended formal from a pure system formal, it's ok. It will (idempotently) stay a pure system formal.

In[90]:=
```
SymbolInspector[Evaluate@CreateExtendedFormal[k]]
```

Out[90]=

| Property | Value |
|---|---|
| Own Value | None (Undefined) |
| Points To | N/A (Literal) |
| Context | System` |
| Character Codes | {63 498} |
| Classification | Pure System |
| Attributes | {Protected} |
| Definition Type | No Definitions |

DO THIS: You can construct extended formals from strings:

In[91]:=
```
SymbolInspector[Evaluate@CreateExtendedFormal["k42"]]
```

Out[91]=

| Property | Value |
|---|---|
| Own Value | None (Undefined) |
| Points To | N/A (Literal) |
| Context | Global` |
| Character Codes | {63 498, 52, 50} |
| Classification | Extended |
| Attributes | {Protected} |
| Definition Type | No Definitions |

DON'T DO THIS: After construction, you can't assign to it:

In[92]:=
```
k42 = 42;
```

Set: Symbol k 42 is Protected.

DO THIS: As usual, the extended formal is now defined until you Clear it:

In[93]:=
```
SymbolInspector[k42]
```

Out[93]=

| Property | Value |
|---|---|
| Own Value | None (Undefined) |
| Points To | N/A (Literal) |
| Context | Global` |
| Character Codes | {63498, 52, 50} |
| Classification | Extended |
| Attributes | {Protected} |
| Definition Type | No Definitions |

## Constructed Symbols in Variables

Works as expected, from strings or from literals

In[94]:=
```
Module[{var = CreateExtendedFormal["m"]}, SymbolInspector[var]]
```

Out[94]=

| Property | Value |
|---|---|
| Own Value | m |
| Points To | m |
| Context | System` |
| Character Codes | {63500} |
| Classification | Pure System |
| Attributes | {Protected} |
| Definition Type | OwnValue (Variable) |

In[95]:=
```
Module[{var = CreateExtendedFormal[j888]}, SymbolInspector[var]]
```

Out[95]=

| Property | Value |
|---|---|
| Own Value | j888 |
| Points To | j888 |
| Context | Global` |
| Character Codes | {63497, 56, 56, 56} |
| Classification | Extended |
| Attributes | {Protected} |
| Definition Type | OwnValue (Variable) |

## On Improper Inputs

We will see $Failed's in some slots, so there's little chance of missing a user error.

In[96]:=
```
Module[{x = CreateExtendedFormal["kj888"]}, SymbolInspector[x]]
```

CreateExtendedFormal: The name 'k j888' is invalid. Extended formals must start with exactly one formal character followed by non–formal characters.

Out[96]=

| Property | Value |
|---|---|
| Own Value | $Failed |
| Points To | $Failed |
| Context | System` |
| Character Codes | {36, 70, 97, 105, 108, 101, 100} |
| Classification | User |
| Attributes | {HoldAll, Protected} |
| Definition Type | OwnValue (Variable) |

In[97]:=
```
Module[{x = CreateExtendedFormal["j888"]}, SymbolInspector[x]]
```

CreateExtendedFormal: The name 'j888' is invalid. Extended formals must start with exactly one formal character followed by non–formal characters.

Out[97]=

| Property | Value |
|---|---|
| Own Value | $Failed |
| Points To | $Failed |
| Context | System` |
| Character Codes | {36, 70, 97, 105, 108, 101, 100} |
| Classification | User |
| Attributes | {HoldAll, Protected} |
| Definition Type | OwnValue (Variable) |

# Master Class in Evaluation Control

Wolfram Mathematica is a term-rewriting system, not a function evaluator like Clojure or JavaScript, let alone a function compiler like C or Java. There are enough similarities between term-rewriting and function-calling that we often speak of “calling a function” in Mathematica, but actually that is the unusual case, as with explicit **`Function`** forms or **`foo[#]&`** syntax. Most of the time, we’re applying rules as if by **`ReplaceAll`**: replacing the mentioning of **`SymbolInspector[x]`** with the definition of **`SymbolInspector`** after replacing its argument pattern **`s_Symbol`** with **`x`**.

But wait, we don’t want the value of **`x`** to go in, we want the symbol **`x`** itself to go in. Mathematica aggressively tries to evaluate (replace) every term, every symbol, at every opportunity. That makes it tricky to write a rule (pseudofunction) like **`SymbolInspector`** that tries to inspect a symbol. Holding on to a symbol inside **`SymbolInspector`** is like holding on to a slippery fish that will take any opportunity to get out of your hands and back into the lake!

```
SetAttributes[SymbolInspector, HoldAll]
```

`HoldAll` stops evaluation of arguments before replacing `SymbolInspector` with its definition.

Without this, if you wrote `SymbolInspector[x]` and it so happened that `x` has the value `5`, Mathematica would evaluate `x` before `SymbolInspector` even starts. The fish slips out of your hands and back into the lake right away. `SymbolInspector` would receive `5`, fail to match `s_Symbol`, and return `$Failed`.

But with the `HoldAll` (or `HoldFirst`) attribute, Mathematica skips argument-evaluation and binds the variable `s` in the body of `SymbolInspector` to the symbol `x`, not to its value `5`.

```
ownVals = OwnValues[Unevaluated[s]];
```

Pass the wriggling fish `s` to `OwnValues` without triggering evaluation.

Even though `s` is held by `SymbolInspector`, passing `s` to another function (really a rule) exposes `s` to the evaluator again. The fish slips away whenever you try to move it. If `s` is `x` and `x = 5`, `OwnValues[s]` becomes `OwnValues[5]`.

Why not `OwnValues[Hold[s]]`? `OwnValues` expects a `Symbol` argument, not a `Hold` object.

`Unevaluated[s]` tells `OwnValues`: "Pretend I passed this fish wrapped in `Hold`, but treat it as the raw argument." `Unevaluated` evaporates after the rewrite of `OwnValues[s]` to its definition.

## The Evaluation Override: `Hold[Evaluate[...]]`

**Task**: Perform a calculation within a metaphorical gated box.

**Scenario**: You have `ownVals`, a list of rules. You need to extract the target symbol, e.g., `x`, from that list and wrap it in a `Hold` so it doesn't evaluate to `5`.

**Mechanics**:

`Hold[...]` normally stops everything.

`Evaluate[...]` exceptionally and idiomatically forces evaluation before the `Hold` freezes the result.

**Result**: `Hold[Evaluate[ownVals[[1,2]]]]` rewrites to `Hold[x]`.

Without `Evaluate`, you would be holding the code `ownVals[[1,2]]` rather than the symbol `x`.

## Trojan Horse: `Replace[heldSym, Hold[sym_] :> With[{...}]]`

**Task**: move a symbol, without triggering evaluation, from the locked box (`Hold`) into a working environment (`With`).

**Gotcha**: You have `heldSym`, which is `Hold[x]`. You can't just say `sym = ReleaseHold[heldSym]` because x would instantly evaluate into 5. The fish wriggles out of the box and back into the lake.

**The Trick**:

- Match the pattern `Hold[sym_]`, binding `sym` to the unevaluated content `x` inside the hold.

- :→ (**RuleDelayed**) inserts that raw symbol **x** into the body of **With**.
- This injection happens *structurally*, bypassing the evaluator.

## Scope: `With[{name = SymbolName[Unevaluated[sym]], ...}, ...]`

**Task**: Calculate properties once and freeze them as constants.

**Why**: Inside the **With** body, **sym** is the raw symbol **x**. If we used **sym** directly in the grid without **Unevaluated**, it would evaluate to **5**. Another opportunity for the fish to slip away.

**Strategy**: Construct safe strings (**name**, **ctx**, **attr**) in the **With** header (using **Unevaluated** shields). In the **With** body, every instance of **name** in the **Grid** is replaced with the string "**x**". Strings are safe; they don't evaluate.

## The Display Wrapper: `HoldForm[s]`

**Task**: "Take a snapshot" of the symbol for display purposes.

Difference from **Hold**: **Hold[s]** evaluates to **Hold[x]**. **HoldForm[s]** evaluates to **x** (visually), but it is an inert form, protected from further evaluation. It is purely cosmetic.

## Summary Table

In[98]:=
```
Grid[{{Style["Keyword", Bold], Style["Role", Bold],
    Style["Plain English Translation", Bold]}, {"HoldAll", "Gatekeeper",
    "Don't compute the inputs; let me handle them raw."}, {"Unevaluated", "Shield",
    "Pass this symbol to the function, but don't let it explode into a value."},
   {"Hold", "Cage", "Keep this expression frozen indefinitely."},
   {"Evaluate", "Override",
    "Compute this specific part now, even though you are inside a lockbox."},
   {"Replace", "Injector", "Move the contents of the lockbox
      into a new piece of code without exploding them."},
   {"HoldForm", "Snapshot", "Show the code as it looks, but don't run it."}},
  Frame → All, Alignment → {{Left, Left, Left}, Top},
  Background → {None, {1 → DarkGray}}, Spacings → {1, 1}]
```

Out[98]=

| Keyword | Role | Plain English Translation |
|---|---|---|
| HoldAll | Gatekeeper | Don't compute the inputs; let me handle them raw. |
| Unevaluated | Shield | Pass this symbol to the function, but don't let it explode into a value. |
| Hold | Cage | Keep this expression frozen indefinitely. |
| Evaluate | Override | Compute this specific part now, even though you are inside a lockbox. |
| Replace | Injector | Move the contents of the lockbox into a new piece of code without exploding them. |
| HoldForm | Snapshot | Show the code as it looks, but don't run it. |